# Soc Home LAB

September 2025

## 1 Wazuh SOC Lab

This project demonstrates the end-to-end deployment and configuration of different security tools using the open-source Wazuh SIEM/XDR platform, Suricata IDS and Pfsense firewall.

## 2 Lab Architecture

The lab is built using VMware VMs and includes the following components:

- **Wazuh Server:** Runs the central Wazuh Manager, Indexer, and Dashboard. Collects and correlates logs from agents, Suricata, and pfSense.
- **Windows Endpoint (Windows 11 Pro):** Runs the Wazuh Agent for system monitoring and log forwarding.
- **Attacker Machine (Kali Linux):** instance used to simulate threats.
- **pfSense Firewall:** Provides firewall logs. Integrated into Wazuh for anomaly detection.
- **Suricata IDS/IPS:** Monitors network traffic, Sends IDS alerts to Wazuh.

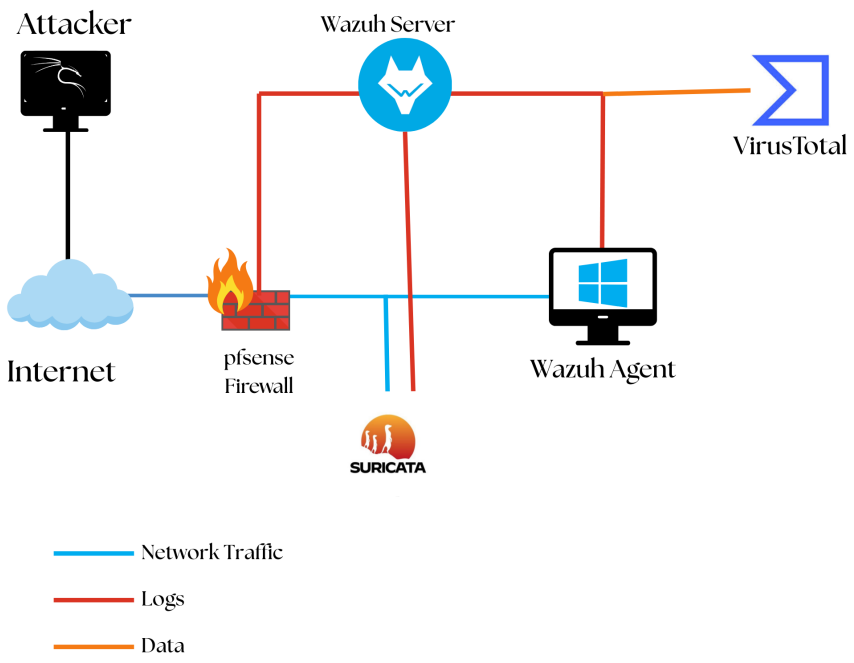



Figure 1: The Lab Architecture

# 3 What is SIEM

**SIEM (Security Information and Event Management)** is a cybersecurity solution that collects, aggregates, and analyzes large volumes of data from applications, servers, network devices, and users. This data typically includes log files, alerts, and metadata.

SIEM plays a crucial role within a **SOC (Security Operations Center)** by enabling:

- Real-time monitoring of network and system activities.
- Collection and interpretation of data from various sources across an organization.
- Detection of security threats and triggering alerts.

Among the various SIEM solutions available on the market, we are focusing on Wazuh as our chosen platform.

# 4 Overview of Wazuh SIEM

Wazuh is an open-source SIEM solution that offers:

- Threat detection
- Integrity monitoring
- Incident response
- Compliance reporting

While there are many SIEM products available (e.g., Splunk, QRadar, Arc-Sight), Wazuh is highlighted here due to its open-source nature and ease of deployment. These qualities make it an attractive choice for organizations seeking a cost-effective and customizable solution to enhance their security operations.

# 5 Wazuh Architecture

The **Wazuh architecture** consists of several integrated components designed for efficient security monitoring and analysis. At the core are **Wazuh agents**, lightweight programs installed on monitored devices that collect security data and send it to a central **Wazuh server**. These agents support a wide range of operating systems, including Linux, Windows, macOS.

The Wazuh server receives this data, decodes and analyzes it using predefined rules, and generates alerts accordingly. These alerts are then forwarded to the **Wazuh indexer**, a scalable search and analytics engine (based on Elasticsearch or OpenSearch) that indexes and stores the information for fast querying and visualization.

The system is complemented by the **Wazuh Dashboard**, a web-based interface built on Kibana that provides visual insights, real-time monitoring, compliance tracking, and investigative tools, making it easier for analysts to detect, respond to, and manage security incidents.

# 6 Wazuh Installation

Wazuh provides an `.ova` file that simplifies deployment using virtualization software. You can download the file from the official Wazuh documentation page:

- `https://documentation.wazuh.com/current/deployment-options/virtual-machine/virtual-machine.html`

# 7 Configuration

- **Network Mode**: NAT (Network Address Translation)
- **Host Platform**: VMware
- **Target Agent OS**: Windows

## Step 1: Import the OVA File into VMware

1. Open VMware Workstation or Player.
2. Import the downloaded `.ova` file.
3. Start the virtual machine.

## Step 2: Access the Virtual Machine

1. Log in with the default username and password:

```
Username: admin
Password: admin
```

2. Open a terminal and run either of the following command to get the IP address:

```
ifconfig
```

3. In your browser's URL bar, enter:

```
https://<IP_ADDRESS>
```

4. The Wazuh login page should open.

## Wazuh Dashboard Interface

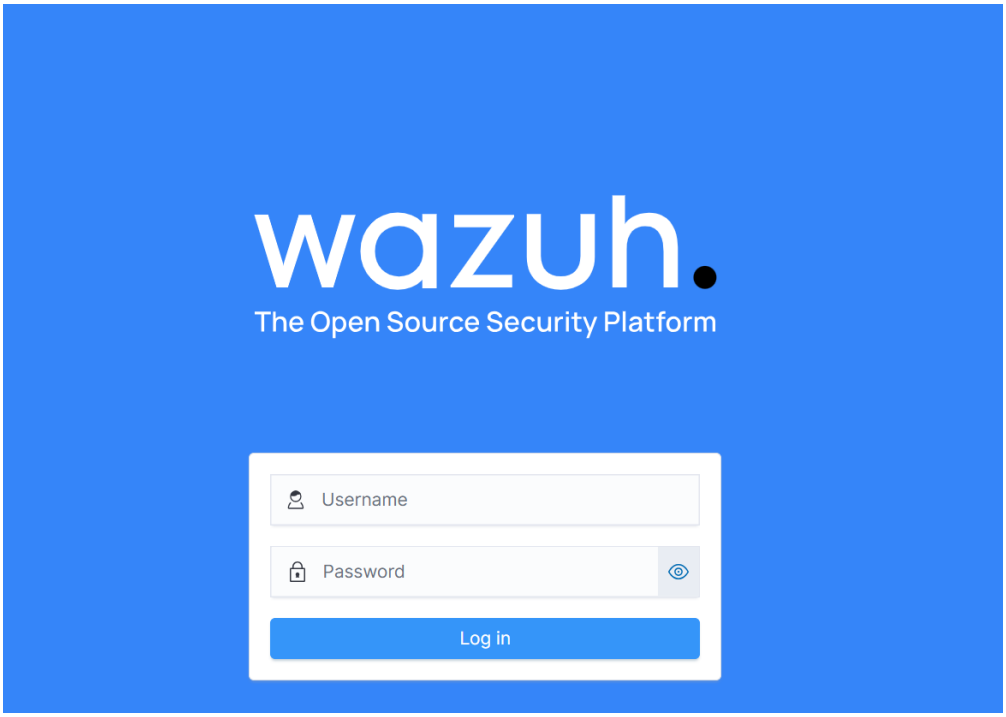



Figure 2: Wazuh Dashboard interface after successful login

## Step 3: Troubleshooting and Services

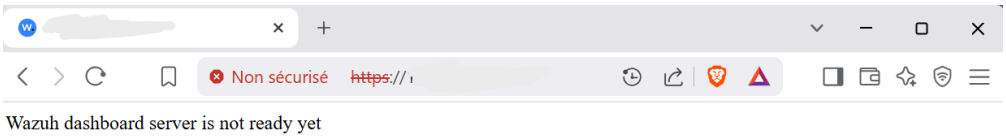



Figure 3: Wazuh Dashboard error: Server not ready

**Dashboard Server Not Ready**

1. Gain Root Privileges

```
sudo su
```

2. Check and Start Wazuh Services

```
systemctl status wazuh-manager
```

**Start it if inactive:**

```
systemctl start wazuh-manager
```

```
systemctl status wazuh-dashboard
```

**Start it if inactive:**

```
systemctl start wazuh-dashboard
```

```
systemctl status wazuh-indexer
```

**Start it if inactive:**

```
systemctl start wazuh-indexer
```

### Step 4: Access the Dashboard

After confirming all services are running, refresh your browser. You should now be able to access the Wazuh Dashboard.

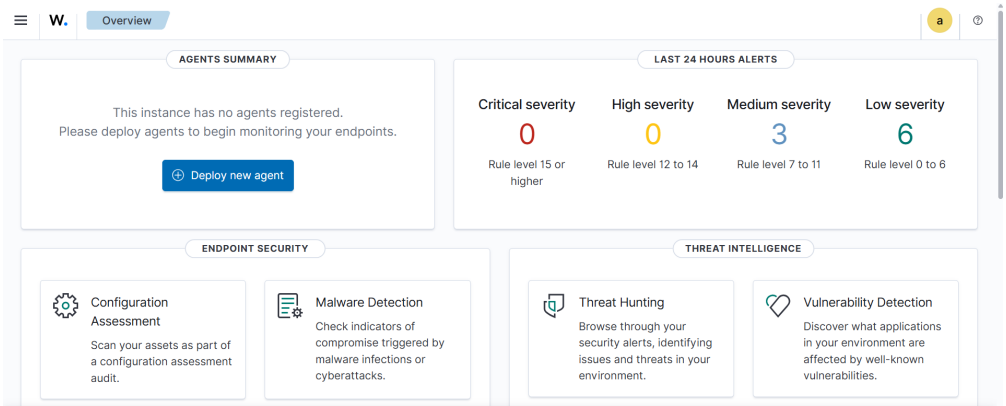



Figure 4: Main Wazuh Dashboard view

# 8 Wazuh Agent Installation and Configuration

## 8.1 What is the Wazuh Agent?

The Wazuh Agent is a lightweight application installed on endpoints such as Windows, Linux, or macOS systems. Its role is to collect security event data and forward it to the Wazuh Manager for processing and analysis. Each agent includes multiple modules such as Log Collection, File Integrity Monitoring (FIM), Security Configuration Assessment (SCA), and Malware Detection, which help identify potential threats and policy violations.

## 8.2 Supported Platforms

- **Windows**
- **Linux**
- **macOS**

## 8.3 Deploying a New Agent from the Wazuh Dashboard

### Step 1: Download the Windows Agent Installer

- Visit the official Wazuh download page: `https://documentation.wazuh.com/current/installation-guide/wazuh-agent/wazuh-agent-package-windows.html`
- Download the Windows `.msi` installer for the Wazuh agent.
- Run the installer to begin the installation process.

### Step 2: Configure Agent Settings

To begin agent installation, access the Wazuh Dashboard:

- Navigate to the `Endpoints` tab.
- Click on `Deploy new agent`.

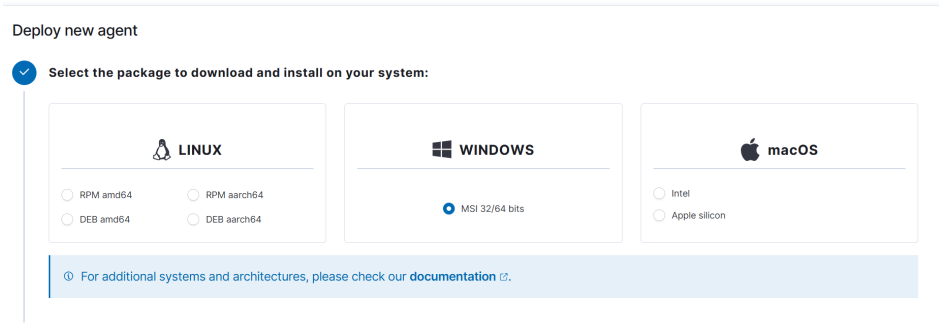



Figure 5: Selecting the Windows agent package

- Enter the **Server Address** — the IP address of your Wazuh Manager.
- Specify a **unique Agent Name** (e.g., `PC1`).

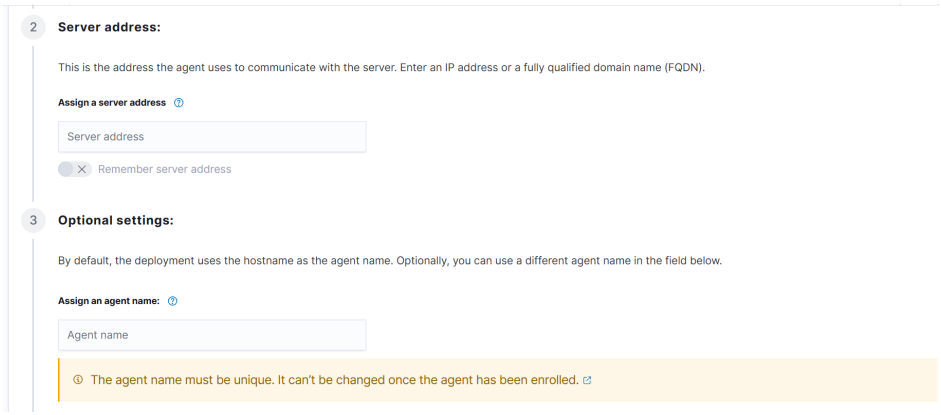



Figure 6: Enter Wazuh Manager IP address and agent name

### Step 3: Register the Agent via PowerShell

After configuration, Wazuh will generate registration commands. These commands must be executed in PowerShell to link the agent with the manager.

- Open **Windows PowerShell** as Administrator.
- Paste and execute the provided commands.

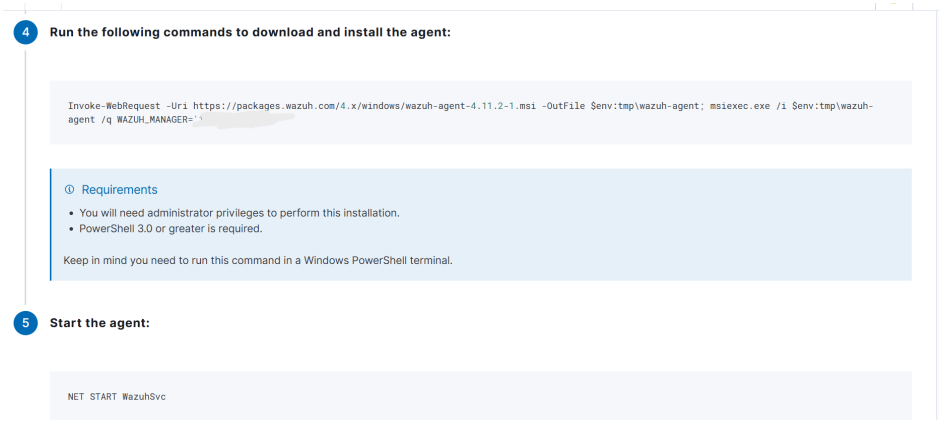



Figure 7: Agent registration — PowerShell (Part 1)

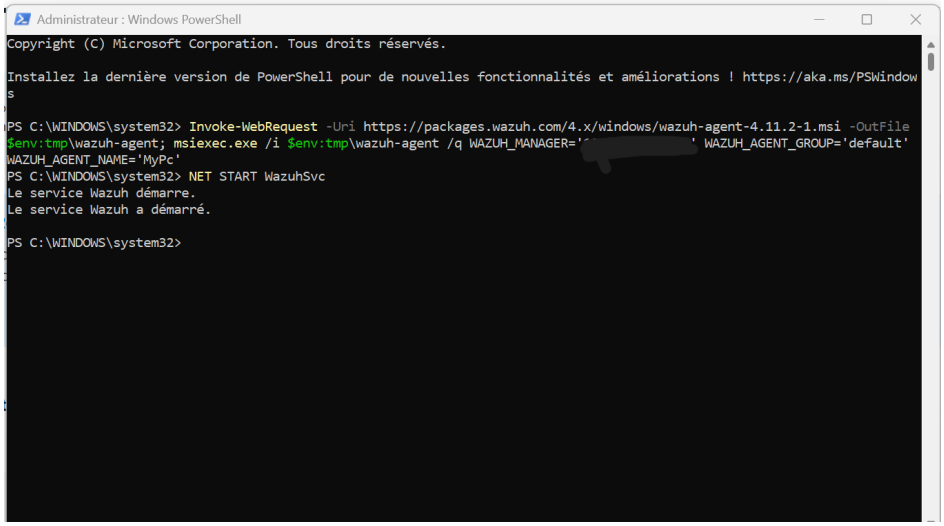



Figure 8: Agent registration — PowerShell (Part 2)

### Step 4: Verify Agent Connection

Once the registration is successful, return to the Wazuh Dashboard.

- Go to the `Agents` section.
- Look for the newly added agent in the list.
- Ensure its status is `Active`.

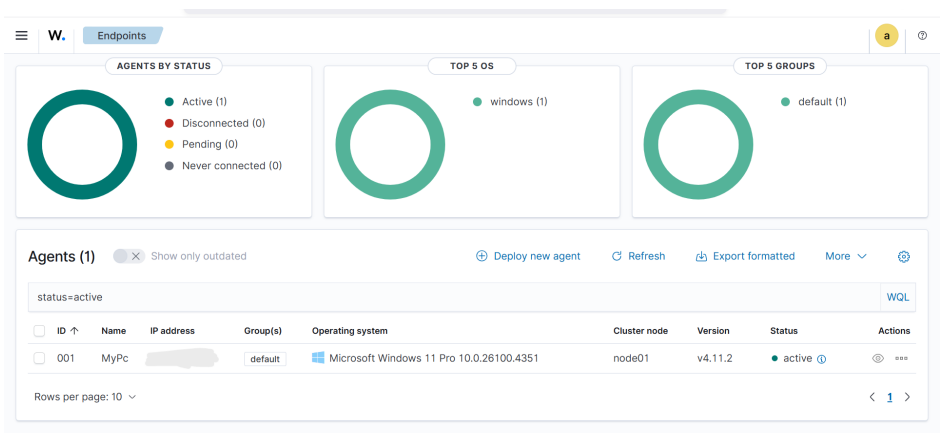



Figure 9: Agent successfully installed and connected

- Once installed, the agent uses a GUI for configuration, opening the log file, and starting or stopping the service.
- By default, all agent files are stored in `C:\Program Files (x86)\ossec-agent` after the installation.

# Pfsense Firewall integration to wazuh

# Introduction

A **firewall** is a critical security solution that inspects and controls incoming and outgoing traffic on a network or device. Think of it like a **security guard at the entrance of a building**: it decides who can enter and who must stay out, based on predefined rules.

In modern environments, firewalls help:

- Prevent unauthorized access
- Block malicious traffic
- Segment networks for better security

There are different types of firewalls: **packet-filtering, stateful inspection, proxy-based, and next-generation firewalls (NGFWs)**, each providing increasing levels of inspection and control.

In this guide, we build a **hands-on SOC lab** using:

- **pfSense** → an open-source firewall and router.
- **Wazuh** → an open-source SIEM for log analysis, alerting, and threat detection.

By integrating pfSense logs into Wazuh, we simulate **real-world SOC operations**, monitoring both network and endpoint events.

# 9 Step 1: Requirements

Before starting, ensure you have the following:

1. **Virtual Machine Software** → VMware Workstation Pro (used in this guide, but VirtualBox also works).
2. **pfSense ISO** → Download the latest version from the official pfSense website.
3. **Wazuh Manager** → Running and accessible in your lab environment.

# 10 Step 2: Creating a Virtual Machine for pfSense

## 10.1 Open VMware

Navigate to **File** → **New Virtual Machine**.

## 10.2 Select the Installation Media

Choose **Installer disc image file (ISO)** and browse for the downloaded pfSense ISO.

## 10.3 Name Your VM

Use a descriptive name such as **pfSense** and pick a save location.

## 10.4 Allocate Disk Space

- **20 GB minimum** (store as a single file recommended).

### 10.5 Customize Hardware

- **RAM**: 1 GB minimum (2 GB recommended).
- **CPU**: 1 vCPU (2 cores recommended).
- **Network Adapters**:
  - **Adapter 1 (WAN)** → NAT (simulates internet access).
  - **Adapter 2 (LAN)** → Host-only (isolated internal lab network).

### 10.6 Assign Interfaces

- **WAN (em0)** → Configured via DHCP (IP in 192.x.x.x/24 range).
- **LAN (em1)** → Static IP 192.168.1.1/24.

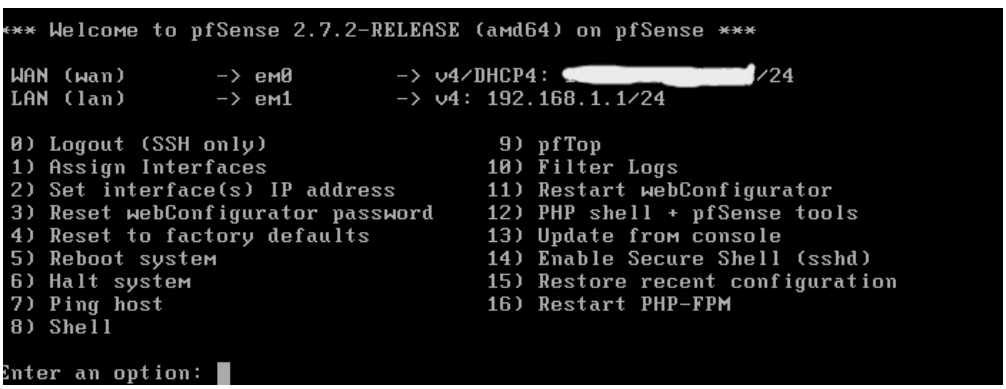



Figure 1: pfSense interface

## 11 Step 3: Accessing the pfSense Web Interface

1. Open a browser from a VM (e.g., Kali Linux) connected to the LAN network.
2. Go to: https://192.168.1.1/
3. Log in with default credentials:
   - Username: `admin`
   - Password: `pfsense`

**Important:** Change the default credentials immediately after login.

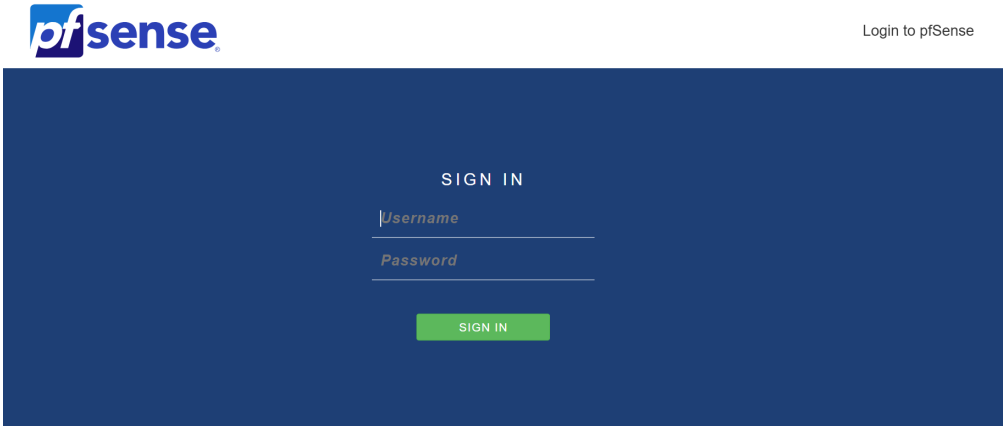


Figure 2: pfSense web interface login

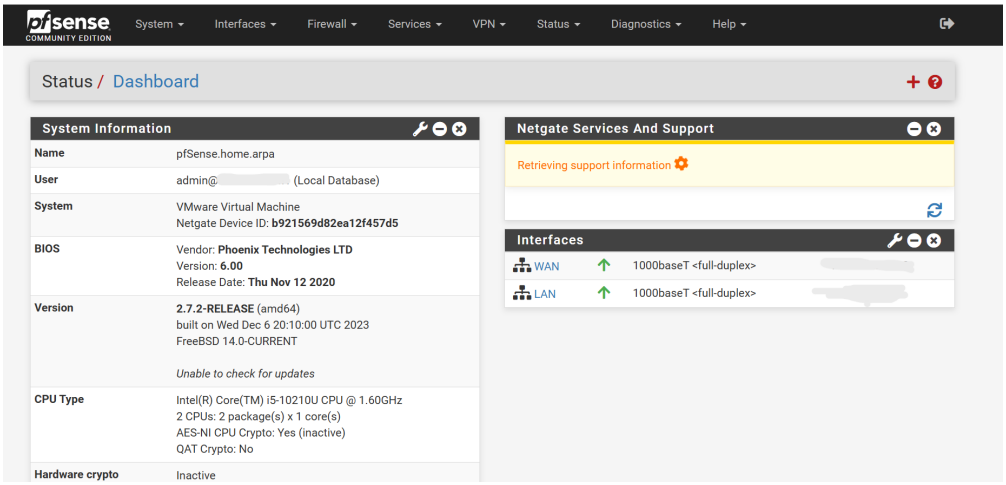



Figure 3: pfSense dashboard

# 12 Step 4: Why Combine pfSense with Wazuh?

- **pfSense** acts as the **network security gateway**, generating logs on traffic, connections, and blocked events.
- **Wazuh** functions as the **Security Information and Event Management (SIEM)** platform, collecting and analyzing logs to detect suspicious activity.

Together, they form a **mini SOC environment**, where firewall events are monitored and correlated with endpoint activity.

# 13 Step 5: Configuring pfSense Logs in Wazuh

## 13.1 Enable Remote Logging in pfSense

- Go to **Status** → **System Logs** → **Settings**.
- Check **Enable Remote Logging**.
- Add your **Wazuh server's IP** (e.g., 192.168.200.50) under **Remote Syslog Servers**.
- Set **Port** to 514 (default syslog).
- Select log categories to forward: System, Firewall, VPN, DHCP, DNS.

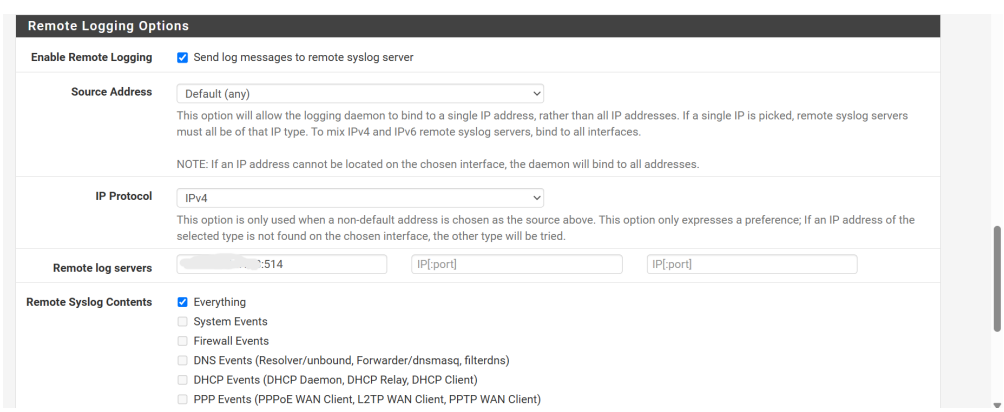


Figure 4: pfSense remote logging configuration

### 13.2 Configure Wazuh Syslog Input

On the Wazuh Manager, edit `/var/ossec/etc/ossec.cfg`:

```
<ossec_config>
  <!-- pfSense syslog input -->
  <remote>
    <connection>syslog</connection>
    <port>514</port>
    <protocol>udp</protocol>
    <allowed-ips>YOUR_PF_SENSE_IP</allowed-ips>
    <local_ip>YOUR_WAZUH_SERVER_IP</local_ip>
  </remote>
</ossec_config>
```

Restart Wazuh after changes.

## 14 Step 6: Creating a Custom Decoder and Rules for pfSense Logs

Since pfSense logs have a unique format, Wazuh may not parse them correctly out of the box.

### 14.1 Custom Decoder

A **decoder** extracts key fields (source IP, destination IP, protocol).

Example (`pfsense-custom-decoder.xml`):

```
<decoder name="pfsense-custom">
  <prematch>filterlog</prematch>
</decoder>

<decoder name="pfsense-fields">
  <parent>pfsense-custom</parent>
  <regex>(\w+)\d+:\S*,\S*,\S*,(\S+),\S*,\S*,(\S+),\S*,\S*,\S*,\S*,\S*,\S*,\S*,\S*,\S*,\S*,(\S+),\S
      *,(\S+),(\S+),(\d+);(\d+),\S*</regex>
  <order>logsource,id,section,protocol,srcip,dstip,srcport,dstport</order>
</decoder>
```

### 14.2 Custom Rules

A **rule** assigns severity levels and alerts based on matched events.

Example (`pfsense-custom-rules.xml`):

```
<group name="pfsense,firewall,">

  <!-- Generic pfSense log -->
  <rule id="100111" level="3">
    <decoded_as>pfsense-custom</decoded_as>
    <description>pfSense log received</description>
  </rule>

  <!-- Successful login -->
  <rule id="100112" level="3">
    <if_sid>100111</if_sid>
    <match>Successful login</match>
    <description>pfSense: Successful login from $(srcip)</description>
```

```
  </rule>

  <!-- Allowed traffic -->
  <rule id="100113" level="5">
    <if_sid>100111</if_sid>
    <match>pass</match>
    <description>pfSense: Allowed traffic from $(srcip) to $(dstip)</description>
  </rule>

  <!-- Blocked traffic -->
  <rule id="100114" level="7">
    <if_sid>100111</if_sid>
    <match>block</match>
    <description>pfSense: Blocked traffic from $(srcip) to $(dstip)</description>
  </rule>

  <!-- Authentication error -->
  <rule id="100115" level="10">
    <if_sid>100111</if_sid>
    <match>authentication error</match>
    <description>pfSense: Authentication error for user $(dstuser) from $(srcip)</description>
  </rule>

</group>
```

- **Level 3 (Informational):** Successful logins.
- **Level 5 (Normal alert):** Allowed traffic.
- **Level 7 (Warning):** Blocked traffic.
- **Level 10 (High):** Authentication errors.

# Interagting Suricata IDS

# Introduction

Intrusion Detection Systems (IDS) and Intrusion Prevention Systems (IPS) are essential components in a layered cybersecurity strategy. These systems inspect network traffic for known and unknown threats to help detect and mitigate attacks.

# 15 IDS vs. IPS Overview

- **IDS (Intrusion Detection System):**
  - Detects threats by inspecting traffic using:
    - Signatures (rules)
    - IoCs (hashes, domains, URLs, TLS/SSH fingerprints)
    - Lua scripting
  - Generates alerts and logs but does not block traffic.
- **IPS (Intrusion Prevention System):**
  - Takes proactive action by dropping or allowing traffic based on inspection results.

# 16 About Suricata

Suricata is an open-source network threat detection engine capable of functioning as both an IDS and an IPS. It is widely used in enterprise and government environments.

- By default, Suricata runs in IDS mode and logs suspicious traffic.
- In IPS mode, it can block malicious packets in real time.

## 16.1 Download and Install

1. Visit: `https://suricata.io/download/`
2. Download the Windows installer (.msi)
3. Run the installer with default settings

Suricata installs to:

```
C:\Program Files\Suricata\
```

Contents include:

- `suricata.exe` – the main executable
- `suricata.yaml` – configuration file
- `rules\` – rule files directory
- `log\` – log output folder

## 16.2 Install and Verify Npcap

Suricata requires Npcap to capture network traffic.

**Step 1:** Download from `https://npcap.com/#download`

During installation:

- Enable WinPcap API-compatible mode
- Enable startup at boot

**Step 2:** Verify that Npcap is running:

```
Get-Service -Name npcap
```

Expected output:

```
Status Name DisplayName
------ ---- -----------
Running npcap Npcap Packet Driver (NPCAP)
```

If not running:

```
Start-Service -Name npcap
```

# 17 Configure Suricata

## Open Configuration

```
C:\Program Files\Suricata\suricata.yaml
```

### Define Network Interface

To view interfaces:

```
Get-NetAdapter | Select Name, Status
```

Example config snippet:

```
af-packet:
  - interface: Wi-Fi
```

### Add Rule Files

In `suricata.yaml`, locate and update:

```
rule-files:
  - local.rules
  - emerging-all.rules
```

Make sure `emerging-all.rules` exists in the `rules\` folder.

### Enable JSON Logging

Enable EVE JSON output:

```
outputs:
  - eve-log:
      enabled: yes
      filetype: regular
      filename: eve.json
```

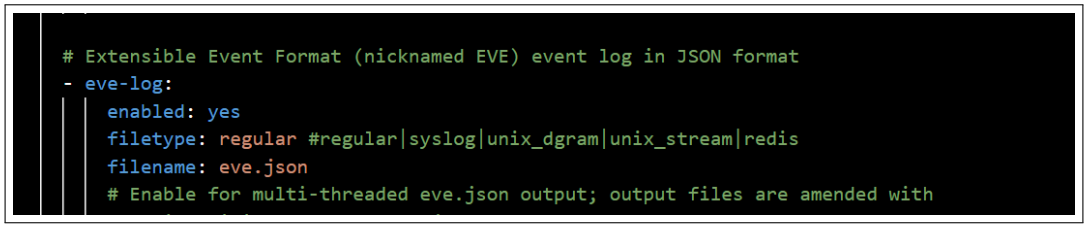



Figure 10: Figure 1: JSON logging configuration in suricata.yaml

## 18 Running Suricata

In PowerShell (Admin):

```
cd "C:\Program Files\Suricata\"
suricata -c suricata.yaml -i <interface>
```

Replace `<interface>` with your adapter name.

### Suricata Log Files

```
C:\Program Files\Suricata\log\
```

Important files:

- `eve.json` – JSON alerts

- `fast.log` – quick alerts
- `stats.log` – system stats

# 19 Integrating with Wazuh SIEM

## Step 1: Confirm EVE JSON Output

Ensure Suricata is writing logs to:

```
C:\Program Files\Suricata\log\eve.json
```

## Step 2: Configure Wazuh Agent

Edit:

```
C:\Program Files (x86)\ossec-agent\ossec.conf
```

Add:

```
<localfile>
  <log_format>json</log_format>
  <location>C:\Program Files\Suricata\log\eve.json</location>
</localfile>
```

Figure 11: Wazuh Agent config for EVE JSON

## Step 3: Restart the Wazuh Agent

```
Restart-Service -Name wazuh
```

## Step 4: Generate Alerts

Use test traffic such as:

```
ping google.com
```

## Step 5: View Alerts in Wazuh

1. Open Wazuh Dashboard
2. Navigate to **Threat Hunting**
3. Filter by `Suricata`

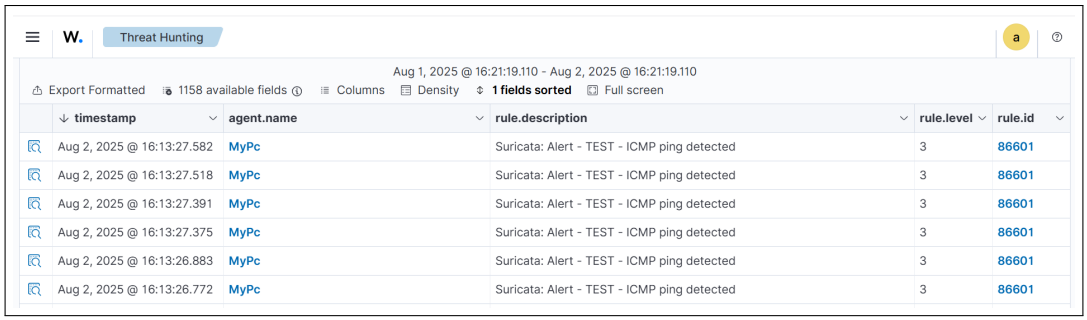



Figure 12: Figure 3: Suricata alert in Wazuh dashboard

## Prerequisites

Before starting, ensure you have the following:

1. A running **Wazuh SIEM** setup
2. At least one **Wazuh agent** connected to the Wazuh manager
3. A valid **VirusTotal API key**

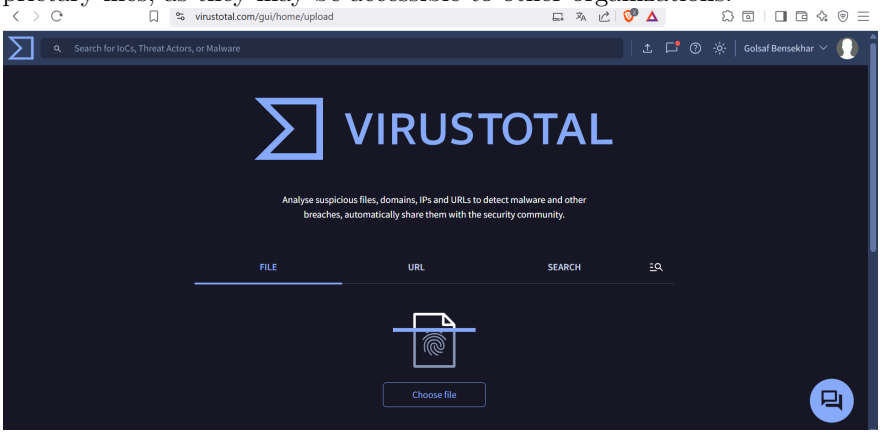
## Interagting VirusTotal for Threat Intelligence

## Introduction

In a modern Security Operations Center (SOC), analysts face hundreds of alerts daily. Without proper context, it can be difficult to distinguish between false positives and real threats. This is where **threat intelligence** becomes essential, it enriches raw alerts with external data, helping analysts prioritize incidents more effectively.

**VirusTotal** is a widely used threat intelligence platform that analyzes files, URLs, domains, and IPs against multiple antivirus engines. By integrating VirusTotal with Wazuh, you can automatically enrich alerts with reputation data, making investigations faster and more accurate.

**Note:** Any file or URL uploaded to VirusTotal should be considered public. Do not upload sensitive or proprietary files, as they may be accessible to other organizations.




## Understanding VirusTotal

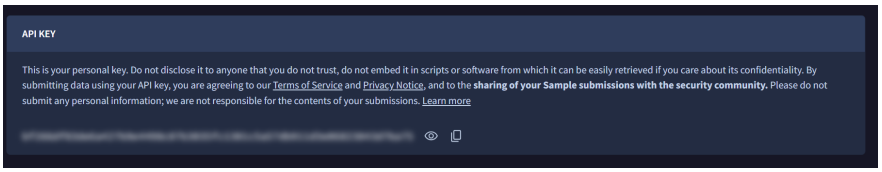
VirusTotal allows security professionals to:

- **Upload and scan files or URLs** using dozens of antivirus engines
- **Retrieve file and URL hashes** for malware tracking
- **Extract metadata** to aid in investigations
- **Leverage the VirusTotal API** to automate lookups and integrate with SIEM platforms like Wazuh

This integration enables Wazuh to automatically query VirusTotal when new files are detected on endpoints, enriching alerts with reputation scores.

## Step 1: Obtain a VirusTotal API Key

1. Create an account on VirusTotal.
2. Navigate to **Profile → API Key**.
3. Copy your personal API key for later use.




## Step 2: Configure Wazuh Manager

On your **Wazuh Manager**, edit the main configuration file:

```
Open C:\Program Files (x86)\ossec-agent\ossec.conf
with a text editor (e.g., Notepad) and add the configuration snippet.
```

Insert the following block (replace `YOUR_API_KEY` with your actual key):

```
<integration>
  <name>virustotal</name>
  <api_key>YOUR_API_KEY</api_key>
  <alert_format>json</alert_format>
</integration>
```

Save the file, then restart the Wazuh Manager:

```
net stop wazuhsvc
net start wazuhsvc
```

## Step 3: Configure the Wazuh Agent

On the **Wazuh Agent**, define which directory should be monitored in real-time. Edit the configuration file:

```
Open C:\Program Files (x86)\ossec-agent\ossec.conf
with a text editor (e.g., Notepad) and add the configuration snippet.
```

Add the following snippet:

```
<directories check_all="yes" realtime="yes">/home/virustotaltest</directories>
```

```
<syscheck>
  <directories realtime="yes">/home/virustotaltest</directories>
</syscheck>
```

This ensures that any file added to `/home/virustotaltest` will trigger a VirusTotal lookup.

Restart the agent to apply changes:

```
net stop wazuhsvc
net start wazuhsvc
```

# Inmplementing File Integrity Monitoring

## 20 Introduction

Attackers don't always just break into systems — they change them. For example, ransomware encrypts files, attackers modify logs, and other malicious actions alter system data. Detecting these changes quickly is critical.

**File Integrity Monitoring (FIM)** is a security practice that tracks file and directory modifications in real time and generates alerts when suspicious activity is detected.

FIM is also a key requirement for many regulatory compliance standards, including **PCI-DSS, HIPAA, and ISO 27001**.

This guide demonstrates how to configure **Wazuh's FIM feature** on a Windows endpoint. It assumes Wazuh is already installed and that the Wazuh agent is deployed on the target machine.

## 21 Configuring Wazuh FIM on Windows

This guide assumes that Wazuh is already installed and the agent is deployed on a Windows machine. To monitor specific files or folders with Wazuh's FIM feature:

1. Open the Wazuh agent configuration file on the endpoint:

   ```
   /var/ossec/etc/ossec.conf
   ```

2. Within the `<syscheck>` block, add the directory you want to monitor. Example: monitoring a folder on the Windows desktop.

   ```
   <directories check_all="yes" report_changes="yes" realtime="yes">
       C:\Users\dell\OneDrive\Bureau
   </directories>
   ```

3. (Optional) Control subdirectory monitoring with the **recursion level** attribute:

   - 0 – monitor only the specified folder
   - 1 – monitor the folder and immediate subfolders
   - 2 – monitor the folder and all nested subfolders up to two levels deep

4. Save the file and restart the Wazuh agent:

   ```
   sudo systemctl restart wazuh-agent
   ```

## 22 Validation: Monitoring in Action

After configuration, Wazuh will begin tracking file activity in the specified path.

As shown in Figure 13 and Figure 14, the test file `file.docx` was:

- Created
- Modified several times
- Deleted

Each of these actions was detected and reported by Wazuh, triggering alerts with appropriate rule levels.

```
<!-- File integrity monitoring -->
<syscheck>

  <disabled>no</disabled>

  <!-- Frequency that syscheck is executed default every 12 hours -->
  <frequency>43200</frequency>

  <!-- Default files to be monitored. -->
  <directories check_all="yes" report_changes="yes" realtime="yes">C:\Users\dell\OneDrive\Bureau</directories>
  <directories recursion_level="0" restrict="regedit.exe$|system.ini$|win.ini$">%WINDIR%</directories>

  <directories recursion_level="0" restrict="at.exe$|attrib.exe$|cacls.exe$|cmd.exe$|eventcreate.exe$|ftp.exe$|lsass.exe$|net.e
  <directories recursion_level="0">%WINDIR%\SysNative\drivers\etc</directories>
  <directories recursion_level="0" restrict="WMIC.exe$">%WINDIR%\SysNative\wbem</directories>
  <directories recursion_level="0" restrict="powershell.exe$">%WINDIR%\SysNative\WindowsPowerShell\v1.0</directories>
  <directories recursion_level="0" restrict="winrm.vbs$">%WINDIR%\SysNative</directories>
```

Figure 13: Configuration to monitor Desktop folder in ossec.conf

**1,521** hits

Jul 31, 2025 @ 16:32:44.272 - Aug 1, 2025 @ 16:32:44.273

Export Formatted | 1133 available fields | Columns | Density | **1 fields sorted** | Full screen

| timestamp | agent.name | syscheck.path | syscheck.event | rule.des... | rule.leve |
|---|---|---|---|---|---|
| Aug 1, 2025 @ 16:32:30.053 | MyPc | c:\users\dell\onedrive\bureau\test file.docx | deleted | File deleted. | 7 |
| Aug 1, 2025 @ 16:31:36.481 | MyPc | c:\users\dell\onedrive\bureau\test file.docx | modified | Integrity ch... | 7 |
| Aug 1, 2025 @ 16:31:36.426 | MyPc | c:\users\dell\onedrive\bureau\test file.docx | modified | Integrity ch... | 7 |
| Aug 1, 2025 @ 16:30:30.325 | MyPc | c:\users\dell\onedrive\bureau\test file.docx | added | File added ... | 5 |

Figure 14: Wazuh alerts showing file creation, modification, and deletion

# Logs and Sysmon Ingestion

## 23 Introduction

Logs are fundamental to visibility, monitoring, and response.

They represent everything that happens in a **system, application, or network**, providing traceability into **what happened, when it happened, and how it happened.** In Windows environments, the **Event Log subsystem** is a primary source of this visibility.

## 24 Logs and Events

- **Log**: A timestamped record automatically generated by a system, application, service, or security tool.
- **Event**: A single instance of an action.
- **Log vs Event**: A log is the written record of an event.

# 25 Importance of Windows Event Logs

Windows Event Logs are one of the pillars of visibility in Microsoft environments.

They provide structured and searchable information on system, application, and security activities.

# 26 Categories of Windows Event Logs

## 26.1 Core Categories

- **System Logs**: Events logged by the operating system and its core components.
- **Application Logs**: Events related to installed applications and software.
- **Security Logs**: Authentication attempts, access control, and other security-related events.

## 26.2 Additional Categories

- **Setup Logs**: Events related to application or system installations.
- **Forwarded Events**: Logs collected from other systems in a centralized logging setup.

# 27 Accessing Windows Event Logs

Windows Event Logs are stored in `.evtx` files in:

`C:\Windows\System32\Winevt\Logs`

## 27.1 Methods of Access

1. **Using PowerShell**

   Example:

   ```
   Get-EventLog -LogName Application
   ```

   *Note: To access Security Logs, PowerShell must be run as Administrator.*

2. **Using the Event Viewer GUI**

   Open via `eventvwr.msc` to browse and filter logs.

# 28 Key Components of a Log Entry

Each log entry contains critical details for investigation:

- **Timestamp**: Exact date and time of the event. Accurate timestamps are critical for detecting, investigating and reconstructing incidents.
- **Event ID**: Numerical identifier for the event type.
- **Message/Description**: Human-readable explanation of what occurred.
- **Entry Type (Event Level):**
  - Information
  - Warning
  - Error
  - Critical
  - Success/Failure Audit

# 29 Event IDs: Identifying Activities

Event IDs allow quick identification of system activities and potential security issues.

## 29.1 User Account Activities

- 4720 — User account created.
- 4726 — User account deleted.
- 4741 — User account changed.

## 29.2 Logon Activities

- 4624 — Successful logon.
- 4625 — Failed logon attempt.

## 29.3 Remote Access and Scripting

- 4104 — Remote PowerShell command executed.

## 29.4 Process and Program Execution

- 4688 — New process created.
- 2003 — USB device connected.

## 29.5 Security and Audit Logs

- 1102 — Security log cleared.

# 30 Centralized Log Collection

When logs are collected across multiple systems, **centralized platforms** (e.g., SIEM solutions) enable:

- Scalable search and filtering
- Correlation of events across systems
- Faster detection and response

Examples: Splunk, Elastic (ELK/Elastic Security), Microsoft Sentinel, Wazuh.

# 31 Sysmon: Enhanced Windows Monitoring

**Sysmon (System Monitor)**: an advanced Windows monitoring tool for collecting and recording detailed system activity.

### 31.1 Installing Sysmon

1. Download from Microsoft Sysinternals.
2. Extract the ZIP.
3. Install with:

```
sysmon.exe -i -accepteula
```

4. Apply a configuration (e.g., SwiftOnSecurity's Sysmon config):

```
sysmon -c sysmonconfig.xml
```

### 31.2 Viewing Sysmon Events

Navigate in Event Viewer:

```
Applications and Services Logs
    Microsoft
        Windows
            Sysmon
                Operational
```

## 32 Ingesting Sysmon into Wazuh

Wazuh uses the **Logcollector module** to collect logs from endpoints, applications, and network devices.

To integrate Sysmon logs into Wazuh:

1. **Update Wazuh Agent Configuration**

```
<localfile>
  <location>Microsoft-Windows-Sysmon/Operational</location>
  <log_format>eventchannel</log_format>
</localfile>
```

2. **Restart the Wazuh Agent**

```
net start wazuh
```

3. **Verify Ingestion**
   - Check the Wazuh dashboard for Sysmon events.
   - Create custom decoders and rules for extracting fields.

# Brute Force Attack: Simulation, Detection  Defense:

## 33 Introduction

A **brute force attack** is a method where an attacker tries multiple password or key combinations repeatedly until the correct one is found. It is one of the simplest yet most common attack techniques used to gain unauthorized access to systems, applications, or accounts.

**Disclaimer:** The following content is for **educational and ethical purposes only**. These techniques should only be executed in a **controlled lab environment** with explicit permission. Unauthorized use against systems you do not own or manage is illegal and punishable by law.

# 34 Types of Brute Force Attacks

- **Simple brute force attack** – Directly attempting all possible combinations of passwords.
- **Dictionary attack** – Using a predefined list of likely passwords (a "dictionary") instead of trying every possible combination.
- **Hybrid brute force attack** – Combining dictionary attacks with variations (e.g., adding numbers or symbols).
- **Reverse brute force attack** – Starting with a single commonly used password and trying it against many usernames.
- **Credential stuffing** – Using leaked username-password pairs from previous data breaches to attempt logins on other systems.

# 35 Popular Brute Force Tools

- **John the Ripper** – A fast password cracker for Unix, Windows, and other platforms.
- **Hashcat** – A powerful GPU-based password recovery tool, ideal for cracking password hashes.
- **Aircrack-ng** – Focused on cracking Wi-Fi network keys.
- **L0phtCrack** – Used to audit and recover Windows passwords.

# 36 Hydra for Brute Force Attacks

**Hydra** is one of the most widely used tools for network login brute forcing. It can perform rapid dictionary-based attacks against more than 50 protocols, including **HTTP, SSH, FTP, RDP, Telnet**, and more.

## 36.1 Hydra Command Structure

```
hydra -L <username_file> -P <password_file> <target>
```

- `-L` : Specifies the file containing a list of usernames.
- `-P` : Specifies the file containing a list of passwords.

# 37 Brute Force SSH

A brute force SSH attack attempts to gain unauthorized access to a remote machine by repeatedly trying different username and password combinations against the **SSH service (port 22 by default)**.

## 37.1 Example Hydra Command

```
hydra -L <username_file> -P <password_file> ssh://<target_ip>
```

# 38 System Requirements

- **Attacker Machine**: Kali Linux
- **Target Machine**: Windows system (with SSH enabled)

  **Note:** Since Windows does not enable SSH by default, you must install and enable OpenSSH Server before running this test.

# 39 Simulating an SSH Brute Force Attack with Hydra

## 39.1 Step 1: Preparing the Password List

On the **attacker machine (Kali Linux)**, we first create a list of possible passwords in a file named `passwords.txt`. This file will be used by Hydra to attempt login combinations against the target system.

Figure 15: Password file used for brute force simulation

## 39.2 Step 2: Running the Hydra Command

Since we already know the username of the target system is `dell`, we use the `-l` option to specify it directly. If we did not know the username, we could use the `-L` option along with a file containing a list of usernames.

```
hydra -l dell -P passwords.txt ssh://<target_ip>
```

Figure 16: Running Hydra against SSH service

## 39.3 Step 3: Observing the Attack

Hydra will automatically attempt multiple login attempts using the given username and password list. Each failed login attempt is logged by the target's SSH service.

# 40 Detection in Wazuh

After simulating the brute force attack, we switch to the **Wazuh dashboard**. In the **Threat Hunting** section, we can see the following indicators:

- Multiple **failed login attempts** for the SSH service.
- Alerts triggered by **Wazuh security rules** detecting brute force activity.
- Event details, such as the source IP (attacker machine) and the affected service (SSH).

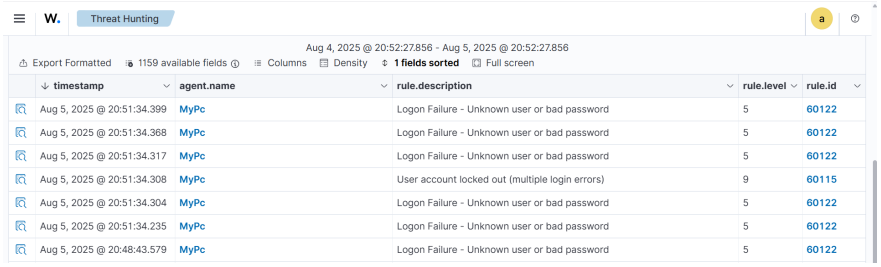



Figure 17: Wazuh detecting brute force attack

# Windows Security Event Analysis

## Event Details

| Field name | Field value | Meaning |
|---|---|---|
| `data.win.eventdata.failureReason` | %%2313 | Unknown user name or bad password. |
| `data.win.eventdata.keyLength` | 0 | No encryption key was used. |
| `data.win.eventdata.logonProcessName` | Advapi | Typical for services, not interactive logins. |
| `data.win.eventdata.logonType` | 8 | Logon type 8 = NetworkCleartext. Credentials were sent in clear text over the network (common with SSHD). |
| `data.win.eventdata.processId` | 0x65d8 | The process ID (0x65d8) of the process that attempted the logon. |
| `data.win.eventdata.processName` | `C:\Windows\System32\OpenSSH\sshd.exe` | Confirms someone tried to connect to this machine over SSH. |
| `data.win.eventdata.status` | 0xc000006d | Status = Logon failure. |
| `data.win.eventdata.subStatus` | 0xc0000064 | User logon with unknown user name OR bad password. |
| `data.win.system.channel` | Security | The event comes from the Windows Security log. |
| `data.win.system.eventID` | 4625 | Event ID 4625 = An account failed to log on. |
| `data.win.system.eventRecordID` | 919137 | A unique sequential ID for this event in the event log. |
| `data.win.system.systemTime` | 2025-08-05T20:51:34.3997254Z | Event timestamp (UTC). |
| `decoder.name` | windows_eventchannel | Wazuh parsed this as a Windows EventChannel log. |
| `input.type` | log | Confirms this came from a log source (Windows logs). |
| `rule.firedtimes` | 14 | At least 14 failed logins of this type have occurred. |

Table 1: Windows Event 4625 – Failed Logon Attempt

## Summary

- On **05-08-2025 at 20:51:34 UTC**, the Windows machine **MyPc** received an SSH login attempt via **sshd.exe**.
- Authentication failed with error **Unknown username or bad password** (`0xc0000064`).
- This was logged as **Event ID 4625** (failed logon).

- Wazuh classified it under **failed authentication events**, severity level **5**, and mapped it to multiple compliance frameworks.
- The rule has already fired **14 times**, so this is **not an isolated event**.

# 41 Defensive Countermeasures

While detection is critical, **prevention** strengthens system defenses against brute force attempts. Recommended strategies include:

- Use strong and unique passwords.
- Enable multi-factor authentication (MFA).
- Regularly monitor login activity.
- Implement rate-limiting to restrict login attempts and lock accounts after repeated failures.